



UNIDADE III

Engenharia de *Software* II

Profa. MSc. Priscila Faccioli

Verificação e Validação de *Software*

- Garante que o produto seja construído corretamente, para que os esforços de correção de problemas causem o menor impacto possível.

As técnicas de V&V abrangem os cenários:

- Aplicação de ferramentas que podem automatizar a revisão de determinados produtos;
- Utilização do processo de revisão de artefatos, como revisão por pares;
- Adoção de normas e padrões;

Manutenção de registros de todas as alterações realizadas nos artefatos.

Processo de medição para avaliar se a qualidade está se mantendo, melhorando ou piorando.

Verificação e Validação de *Software* (V&V)

Aspectos para levar em conta na utilização do V&V:

- Permite encontrar erros mais cedo;
 - Aumenta a integração da equipe;
 - Permite o acompanhamento contínuo da qualidade;
 - Facilita o gerenciamento;
 - Melhora a qualidade;
 - Aumenta a interação das equipes;
-
- Deve ser aplicado durante todo o ciclo de desenvolvimento e do planejamento das demais atividades;
 - A ausência do planejamento limita a qualidade;
 - Gera custos mais elevados para encontrar falhas introduzidas no início do desenvolvimento.

Verificação

- Consiste nas ações realizadas ao final de cada fase;
- Objetivo é atestar que o produto está sendo desenvolvido corretamente;

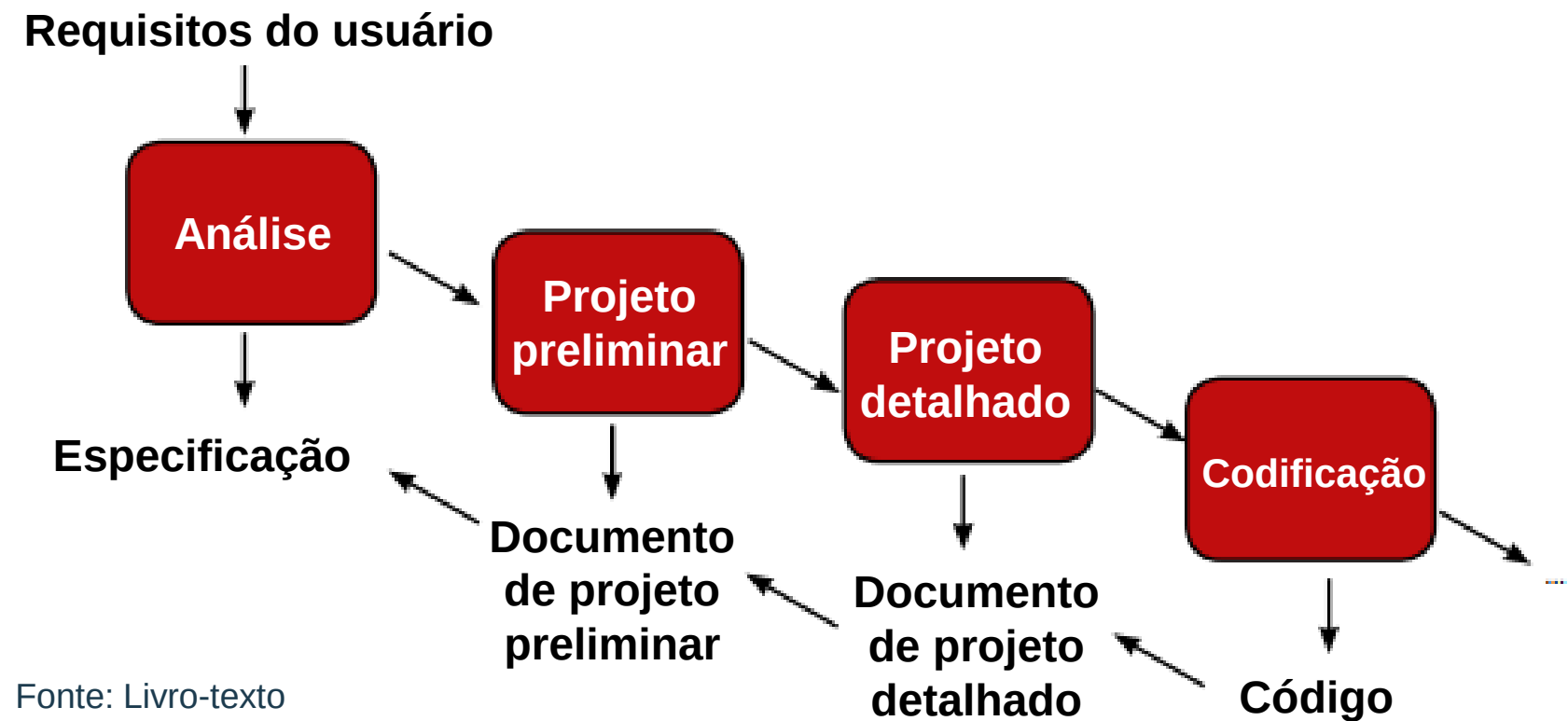


Figura 27 – Atividades de verificação da qualidade

Validação

- Consiste nas ações realizadas ao final ou no decorrer do processo de desenvolvimento;
- Objetivo de avaliar se o produto está de acordo com as especificações de requisitos iniciais fornecidas pelo cliente e
- Garantir que o produto tenha sido desenvolvido corretamente.

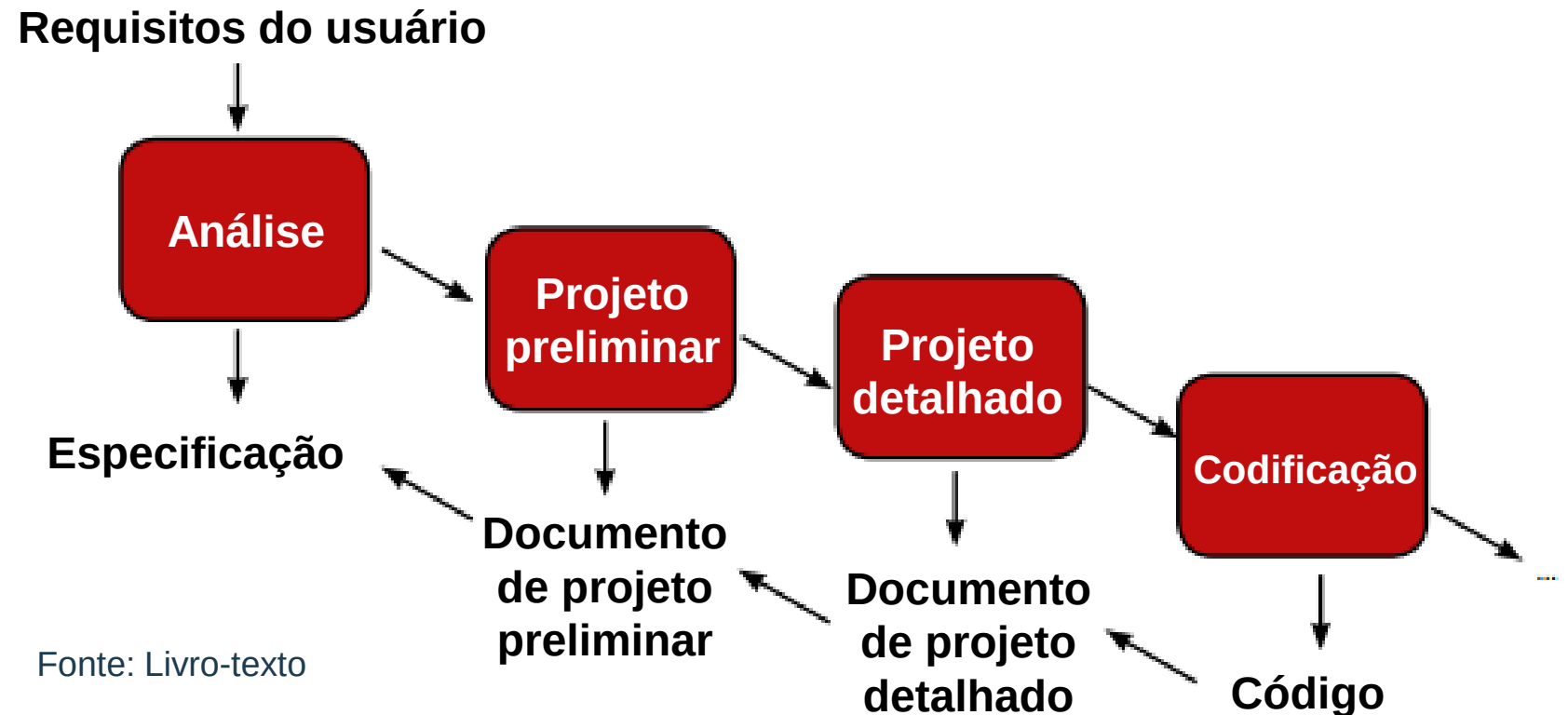


Figura 27 – Atividades de verificação da qualidade

Revisões técnicas

- Avaliação crítica de todos os artefatos produzidos durante o desenvolvimento de um *software*, em pontos predefinidos do ciclo de vida;
- Objetivo: Encontrar e corrigir erros inseridos durante o processo.
- Quanto mais cedo encontrarmos esses erros, teremos redução de custos no processo de desenvolvimento.

Benefícios:

- Verificação se o produto de trabalho está em conformidade com os padrões, as especificações e os requisitos definidos; identificação de melhorias necessárias;
- Documentação e geração de histórico de erros;
- Consenso entre cliente e equipe sobre os produtos de trabalho.

Revisão Técnica Formal (RTF)

- Deve ser estruturada,
- Conduzida em uma reunião e será tão bem-sucedida quanto for planejada e controlada. Deve ser realizada sobre artefatos para permitir melhor avaliação e aumentar as probabilidades de sucesso.



Fonte: Livro-texto

Revisão Técnica Formal (RTF) – Boas práticas

Boas práticas para a condução de uma RTF:

- A equipe de revisão deve ter de três a cinco pessoas;
- Equipe com um objetivo alvo para revisão;
- Artefato a ser revisado deve ser enviado com antecedência;
- Os papéis dos *stakeholders* devem estar claramente definidos: moderador, autor e revisores;
- Iniciar pela leitura do documento pelo autor;
- A cada parte do documento, a equipe de revisores faz os comentários;
 - O líder faz a mediação em eventuais conflitos e discordâncias;
 - Registrar todos os comentários da revisão para histórico;
 - Duração de, no máximo, 2 horas.

Diretrizes básicas de uma RTF

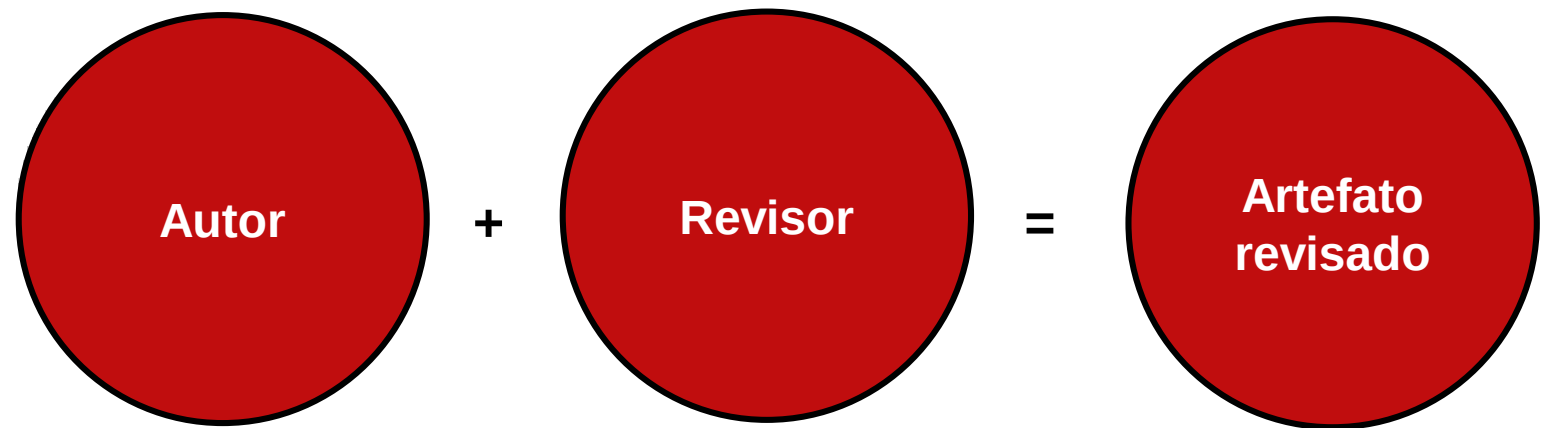
As RTFs são procedimentos relativamente simples que produzem resultados significativos na garantia da qualidade de um produto de *software*.

Diretrizes básicas de condução:

- Fixe e mantenha uma agenda com os participantes;
- Revise o produto, não o autor do artefato;
- Faça anotações por escrito;
- Enuncie os problemas, mas não tente resolver cada um deles;
 - Limite o debate;
 - Realize um treinamento sobre revisões para todos os revisores;
 - Analise suas antigas revisões.

Passeio (*walkthrough*)

- São revisões técnicas informais, também chamados de revisão por pares, mas podem ter até três participantes: autor, revisor e moderador.
- O revisor pode ser um técnico, um cliente ou uma pessoa externa ao projeto que domine o assunto em revisão.
- O moderador não deve ser do mesmo nível hierárquico do autor e do revisor.
- Prática recomendada pelos métodos ágeis de desenvolvimento.
- Apoia a recomendação da qualidade a qual determina que tudo deve ser revisado antes de ser entregue ao cliente.



O processo: Passeio (*walkthrough*)

- Reuniões informais para avaliação dos produtos.
- Extremamente simples;
- Não envolve agendamento, preparação ou planejamento prévio.

Características:

- Pouca preparação requerida;
- Objetivo de comunicar ou receber aprovação do artefato;
- Papéis específicos não são estabelecidos;
- Autor guia os presentes pelo artefato;
- O revisor lê o documento e faz suas considerações;
- O autor nunca pode ser o leitor.

Diretrizes básicas de um *Walkthrough*

Algumas recomendações importantes que devem ser observadas:

- O autor seleciona os revisores e convida-os para a reunião;
- O autor entrega o artefato aos revisores na reunião;
- O foco deve ser o artefato, e não o autor;
- O autor deve apresentar o artefato durante a reunião;
- Os revisores apresentam possíveis falhas, comentários e sugestões de melhoria;
- Com base nas informações apresentadas, o autor faz as devidas correções.

Revisões progressivas por pares

Constituem características da walkthrough e da RTF.

- Artefato é dividido em partes e distribuído aos revisores;
- O artefato deve ser separado e distribuído aos revisores;
- Na revisão, cada revisor faz a leitura do artefato;
- Há o registro das revisões;
- Ao término de cada trecho revisado, o revisor passa a vez para o próximo, e assim sucessivamente, até que todo o material seja revisado;
- O autor já faz as alterações sugeridas para verificar a eficácia destas.

Interatividade

Qual é a técnica de testes que agrupa boas práticas do RTF e do *Walkthrough*?

- a) Validação.
- b) Verificação.
- c) Revisões progressivas por pares.
- d) Teste de Caixa-preta.
- e) Teste Estrutural.

Resposta

Qual é a técnica de testes que agrupa boas práticas do RTF e do *Walkthrough*?

- a) Validação.
- b) Verificação.
- c) Revisões progressivas por pares.
- d) Teste de Caixa-preta.
- e) Teste Estrutural.

Técnica de Inspeção

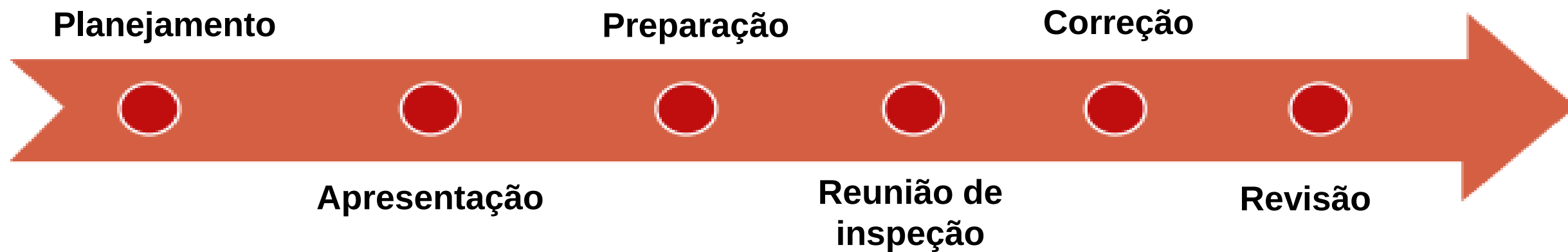
- Técnica formal, em que os envolvidos examinam os artefatos produzidos contra uma especificação inicial com o objetivo de encontrar incoerência e erros;
- Realizada a qualquer momento dentro do ciclo de vida de um produto de *software*;
- Envolve pessoas com domínio do assunto que está sendo inspecionado e possui um *checklist* dos pontos que devem ser verificados no artefato.

Qualquer produto resultado do trabalho de desenvolvimento, inclusive:

- Documento de requisitos;
- Especificação de casos de uso;
- Diagrama de classes;
 - Protótipo de telas;
 - Especificações de telas;
 - Diagrama de arquitetura do sistema;
 - Modelo de dados;
 - Especificação de projeto etc.

Processo de Inspeção

Processo de inspeção é composto de seis etapas, detalhado a seguir:

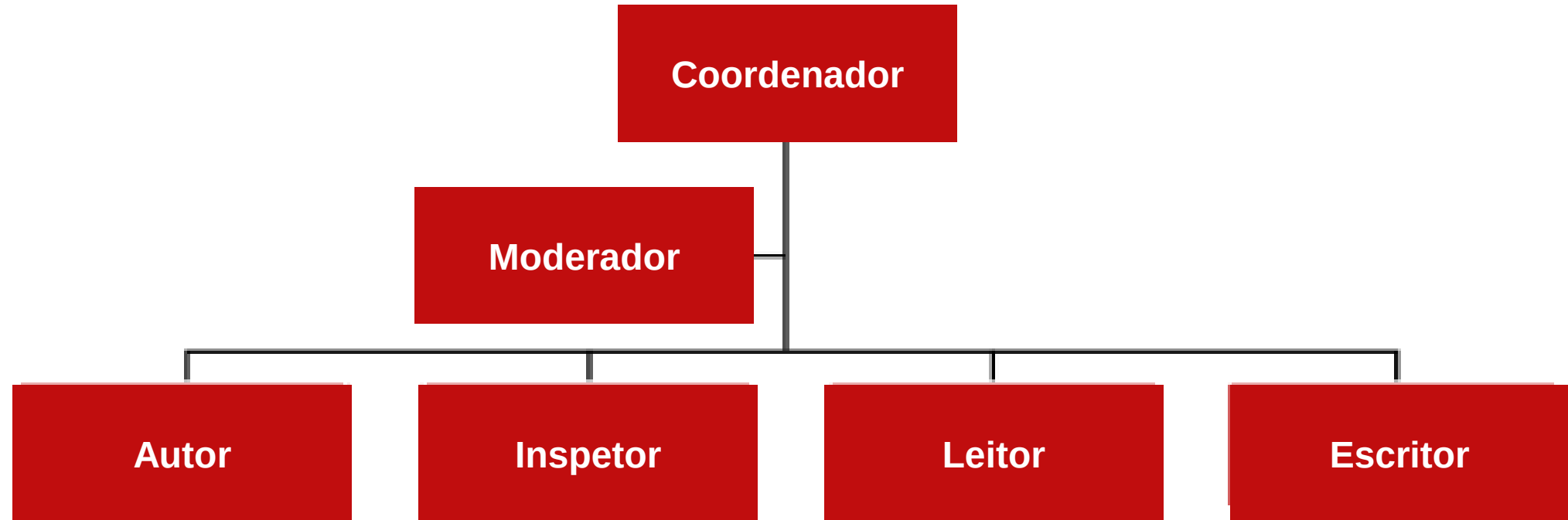


Fonte: Livro-texto

Processo de Inspeção

- Planejamento: Previsão da inspeção dos artefatos no cronograma do processo de desenvolvimento de *software*, sem o qual a formalidade prevista não será atendida.
- Apresentação: Etapa opcional diretamente ligada ao tamanho e à complexidade do artefato a ser inspecionado. No caso de ocorrer, há uma explicação sobre seus pontos principais.
- Preparação: Leitura prévia de toda a documentação dentro do tempo planejado.
- Inspeção: Apresentação do artefato por partes, para que os inspetores façam os questionamentos e apontamentos de correção, que são registrados pelo escritor.
- O processo se repete até que todo o artefato seja inspecionado. Ao final, a equipe faz as considerações de melhoria e decide se o artefato será aceito, aceito com ressalvas ou se necessita de nova inspeção.
 - Correção: As correções e melhorias são realizadas com registro das respectivas soluções.
 - Revisão: Situação final da inspeção. O coordenador de inspeções recebe o documento final da inspeção.

Papéis e Responsabilidades no Processo de Inspeção



Fonte: Livro-texto

Checklist de uma Inspeção para artefatos de software

São listas de verificação elaboradas em informações históricas de outras inspeções, lições aprendidas, dados fornecidos por especialistas e itens retirados de padrões preexistentes contra os quais o artefato será avaliado.

Fonte: Livro-texto

Artefato	Itens de verificação
Especificação dos casos de uso	As precondições foram descritas? As especificações estão de acordo com a tela? Os fluxos alternativos foram corretamente indicados? As regras de negócio foram apontadas? Na descrição dos casos de uso não há referência de navegação? Os atributos de entrada e saída estão descritos? As pós-condições estão explícitas?
Diagrama de classes	As classes identificadas referem-se a objetos reais do sistema? Os relacionamentos de agregação e herança fazem sentido no contexto do sistema? Métodos privados foram identificados? As classes têm menos de vinte atributos? O requisito de baixo acoplamento foi respeitado? O requisito de alta coesão foi atendido?
Diagrama de dados	Todas as tabelas possuem chaves primárias? A chave primária está correta? Existem índices secundários criados? As chaves secundárias estão corretamente relacionadas às tabelas?
Documento de arquitetura	Foram considerados os padrões do cliente? Existe uma implementação do padrão definido para servir de referência? Foram considerados os requisitos de segurança? Foram considerados os requisitos de desempenho? Foram considerados os padrões de implementação da linguagem?
Codificação	As variáveis estão com nomes adequados? Todas as variáveis são inicializadas? Todas as variáveis são do tipo privado? Os métodos estão com nomes adequados? Todos os métodos retornam valores? Condições e <i>loops</i> estão corretamente grafados?

Sala limpa (*Cleanroom*)

- Baseada em especificações formais matemáticas e destinada ao desenvolvimento de *software* de alta confiabilidade como os utilizados no controle de usinas nucleares, na navegação de aviões, trens, metrô e navios, dentre outros.
- O foco da sala limpa é a realização de ações preventivas, e não a correção de erros.
- A verificação se dá por meio de inspeções rigorosas, provas matemáticas e testes estatísticos para determinar a confiabilidade do sistema, tornando-a diferente de outras técnicas (SOMMERVILLE, 2013).

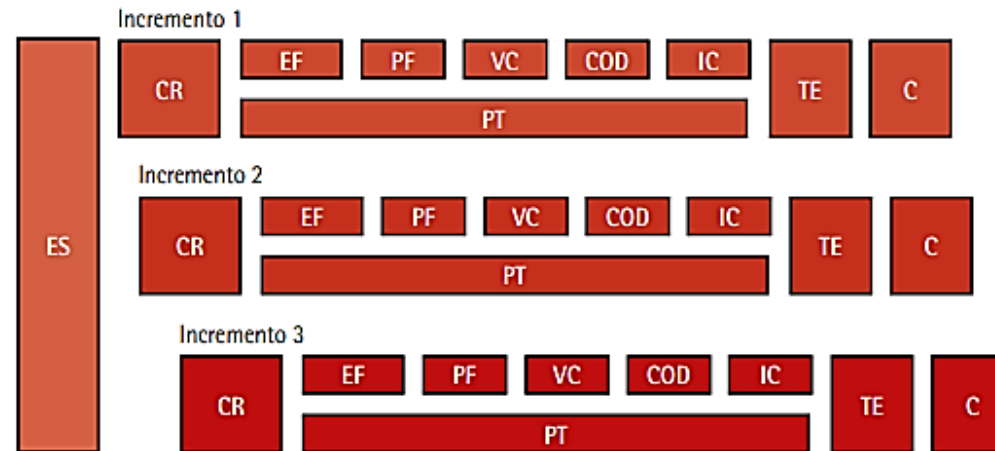
Sala limpa (*Cleanroom*)

Crenças limitantes:

- Metodologia muito teórica, matemática e radical para o desenvolvimento de *software*;
- Substituição de testes unitários por verificação de correção e controle estatístico de qualidade é muito diferente do método empregado pelos desenvolvedores de *software*;
- As empresas de *software* ainda não possuem nível de maturidade suficiente para aplicar o processo sala limpa.

Sala limpa (*Cleanroom*) – Estrutura

- Processo de gerenciamento: define as atividades relacionadas à gestão do projeto.
- Processo de especificação: descreve as atividades de coleta de requisitos, a especificação formal dos requisitos, a especificação formal do projeto etc.
- Processo de desenvolvimento: define as atividades de engenharia relativas à codificação e à inspeção formal do código produzido.
- Processo de certificação: define o uso dos modelos de referência, os testes estatísticos formais, a execução da aplicação e a certificação da confiabilidade do *software* em desenvolvimento.



- COD (geração de código)
- IC (inspeção de código)
- TE (teste estatístico)
- C (certificação)
- PT (planejamento de teste)
- ES (engenharia de sistemas)
- CR (coleta de requisitos)
- EF (especificação formal)
- PF (projeto formal)
- VC (verificação de correção)

Interatividade

Qual etapa do processo de inspeção é opcional?

- a) Revisão.
- b) Correção.
- c) Apresentação.
- d) Inspeção.
- e) Planejamento.

Resposta

Qual etapa do processo de inspeção é opcional?

- a) Revisão.
- b) Correção.
- c) Apresentação.
- d) Inspeção.
- e) Planejamento.

Teste de *Software*

Definições:

- Segundo Myers (2004), testar um *software* é um processo de executar um programa ou sistema com a intenção de encontrar defeitos.
- Para Dijkstra (1970), os testes podem mostrar a presença de falhas em um *software*, mas nunca a sua ausência.
 - Para Hetzel (1988), testes são uma atividade que, a partir da avaliação de um atributo ou capacidade de um programa, torna possível determinar se ele alcança os resultados esperados.

Conceituação de Defeito, Erro e Falha

Termo	Termo em inglês	Definição
Engano	<i>Mistake</i>	Ação humana que produz um resultado incorreto
Falha	<i>Fault</i> ou <i>bug</i>	Manifestação, no <i>software</i> , de um engano cometido pelo desenvolvedor
Erro	<i>Error</i>	Diferença entre o valor obtido e o valor esperado, ou seja, qualquer estado inesperado na execução do <i>software</i>
Defeito	<i>Failure</i>	Incapacidade de o <i>software</i> fornecer o serviço especificado

Fonte: IEEE (1990)

Principais tipos de Defeitos

Fonte: RIOS; MOREIRA (2013)

Tipo de defeito	Definição
Funcionalidade	Quando o <i>software</i> não faz o que o usuário espera que ele faça
Usabilidade	Quando há dificuldade de navegação, a cor do texto está muito clara, dificultando a leitura, ou o conteúdo é muito extenso, obrigando o usuário a usar a barra de rolagem constantemente
Desempenho	O <i>software</i> não atende com a rapidez necessária às solicitações do usuário, especialmente no caso dos sistemas muito interativos
Prevenção de defeitos	O programa não se protege das entradas de dados não previstas, que, posteriormente, são processadas de forma inadequada. O <i>software</i> pode, por exemplo, aceitar valores em branco em campos numéricos
Detecção e recuperação de defeitos	O programa não trata as operações, como: <i>overflow</i> , <i>flags</i> de defeitos, ou trata de forma inadequada
Limites	O <i>software</i> não consegue tratar ou trata inadequadamente valores extremos (o maior, o menor, o primeiro, o último) ou fora dos limites
Cálculo	O <i>software</i> executa um cálculo e produz um resultado errado. Muitas vezes, por questões de aproximação, uma fórmula não produz os resultados esperados
Inicialização	Alguns <i>softwares</i> ou rotinas devem ser inicializados quando usados pela primeira vez ou sempre que são chamados para execução. Exemplo de inicialização de programa: na primeira vez em que executa, o <i>software</i> deve criar um arquivo em disco. A ausência deste arquivo poderá causar problemas nas etapas seguintes do processamento
Condições de disputa	Ocorre quando o <i>software</i> espera pela resposta dos eventos A e B, sendo suposto que A sempre termine primeiro. Se por algum problema B terminar primeiro, o <i>software</i> poderá não estar preparado para esta situação e apresentar resultados inesperados

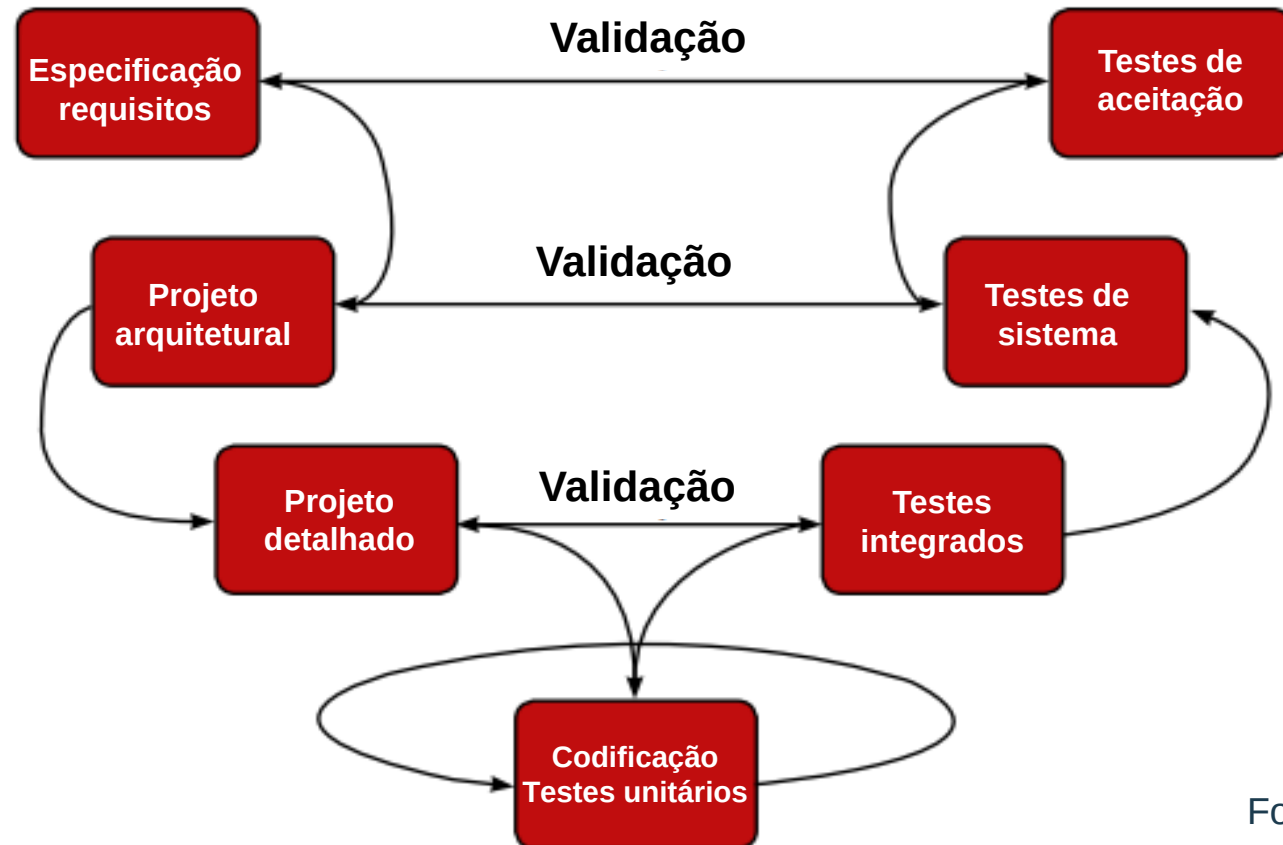
Ciclo de Vida de Testes

Segundo Rios e Moreira (2013), para que o processo de testes seja eficiente, é necessário:

- Entender o sistema em todos os seus detalhes;
- Dominar as técnicas de testes;
- Ter habilidade para aplicar essas técnicas.

Modelo V

- Foi desenvolvido a partir do Cascata,
- Incluir o processo de testes desde o início do ciclo de desenvolvimento do *software*.
- Foco: Permitir a identificação de defeitos o mais corretamente possível, por meio de atividades de verificação e validação e da criação de casos de testes e roteiro de testes.



Roteiro de Testes

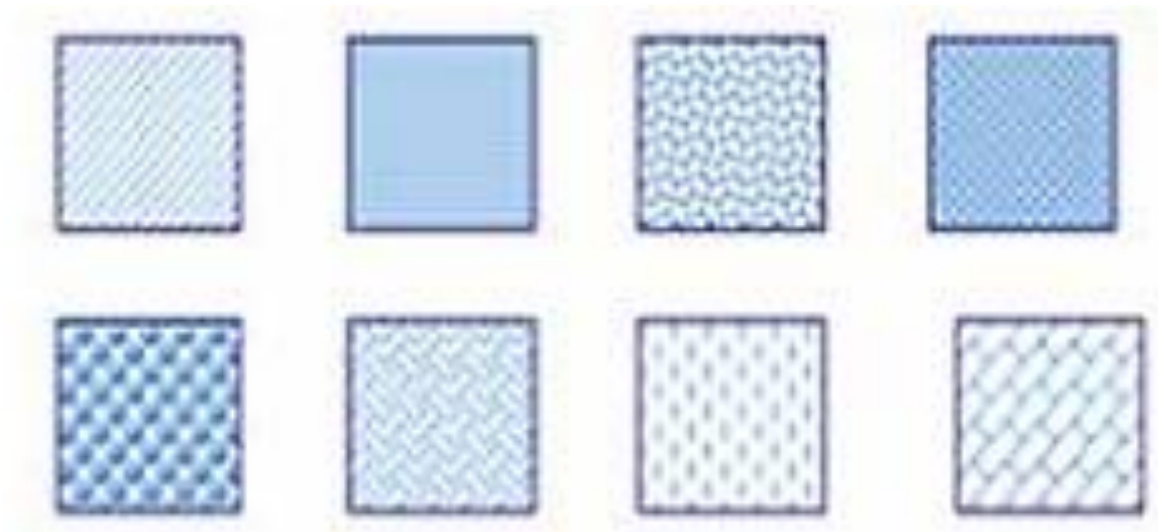
- O roteiro de testes que descreve em detalhes o passo a passo para a realização dos testes, especificando o que deve ser feito e qual o resultado esperado.
- O nível de detalhe deve ser suficientemente claro para ser executado por pessoa externa ao processo.
- Esse roteiro deve ser submetido à inspeção e à aprovação do usuário antes de ser utilizado e deve cobrir todas as situações que os usuários utilizam na fase de aceite do sistema.



Fonte: Livro-texto

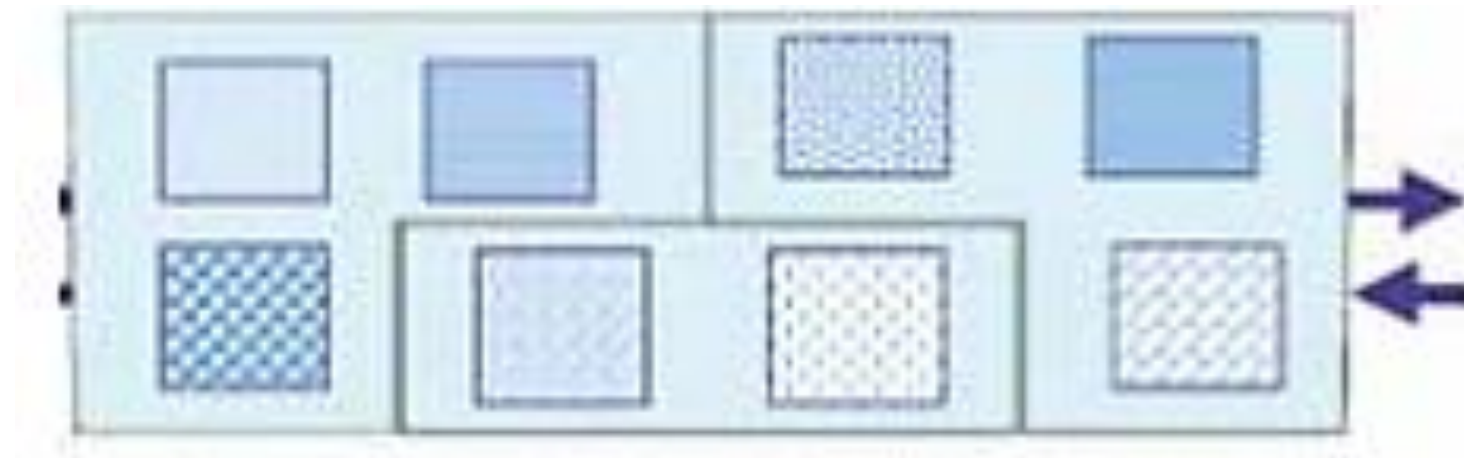
Testes de unidade ou testes unitários

- São testes realizados pelos desenvolvedores de *software*;
- Objetivo: garantir o funcionamento adequado do programa, do módulo, da função ou da classe que foi construída.
- Podem ser automatizados ou realizados por meio de ferramentas.



Testes de integração

- Realizados após o teste unitário para garantir que os elementos que compõem a aplicação funcionem de forma integrada com sucesso.
- Realizados pelos analistas de sistemas, envolvem testes de subsistemas ou incrementos do *software*.



Fonte: Livro-texto

Testes de Sistemas, Aceitação e Regressão

Teste de Sistemas:

- Verificam se a aplicação desenvolvida está de acordo com a especificação inicial do sistema.
- O ambiente deve ser semelhante ao ambiente de produção.

Teste de Aceitação:

- Realizados pelos usuários finais e analistas de testes;
 - Garantem que todos os requisitos solicitados foram incluídos e funcionam corretamente no produto entregue.
-
- São feitos utilizando os critérios estabelecidos pelos usuários como se estivesse utilizando o *software* no seu dia a dia.
 - Teste de Regressão: Quando há manutenção do sistema, toda a aplicação deve ser testada para garantir que nenhuma rotina que tenha sido afetada pela mudança contenha erros.

Testes Funcionais (Caixa-preta) e Testes Estruturais (Caixa-branca)

Teste de Funcionais (Caixa-preta):

- Envolve o correto funcionamento do *software*, sua integração com outros sistemas, suas interfaces e a garantia de que qualquer alteração feita não afete a aplicação.

Teste Estrutural (Caixa-branca):

- Avalia a qualidade do código produzido pelos desenvolvedores, garantindo que toda linha de código escrita seja executada pelo menos uma vez e não contenha erros.

Testes: Caminho independente e Complexidade ciclômática – $V(G)$

Complexidade ciclômática $V(G)$: Determina a quantidade de testes necessários para garantir que todas as linhas de código sejam executadas pelo menos uma vez.

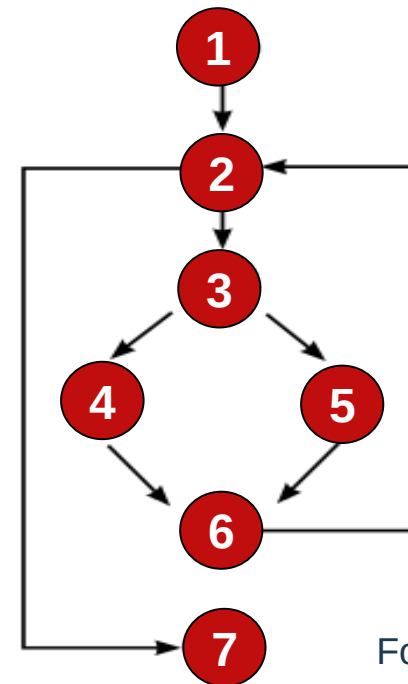
Formas para se calcular: $V(G) = (E - N) + 2$,

em que: E é o número de arestas.

N é o número de nós do grafo.

$V(G) = P + 1$, em que: P é o número de nós predicados do grafo.

Teste de Caminho independente: Contém pelo menos uma nova aresta no grafo de controle e garante que todo comando será executado pelo menos uma vez.



Caminhos independentes

1-2-7

1-2-3-4-6-2

1-2-3-5-6-2

Fonte: Livro-texto

Interatividade

Qual é o tipo de teste que preocupa-se com os requisitos funcionais sem a necessidade de debugar o código fonte ?

- a) Caixa-branca.
- b) Caixa-preta.
- c) Teste Unitário.
- d) Complexidade ciclomática.
- e) Teste de regressão.

Resposta

Qual é o tipo de teste que preocupa-se com os requisitos funcionais sem a necessidade de debugar o código fonte ?

- a) Caixa-branca.
- b) Caixa-preta.
- c) Teste Unitário.
- d) Complexidade ciclomática.
- e) Teste de regressão.

Casos de Testes

- Objetivo: Situação que o sistema apresenta para a qual, dada uma informação de entrada, será gerada uma saída esperada pelo usuário.

Exemplo: Testar *software* para uma calculadora simples com quatro operações. Todas as operações precisam ser testadas, e um conjunto de valores precisa ser verificado:

Nessa situação, os casos de testes para operação de soma seriam:

- $2 + 2 = 4$ – soma de dois números inteiros positivos.
- $-2 + 2 = 0$ – soma de número inteiro positivo e negativo.
- $-2 - 2 = -4$ – soma de dois números.
- $2 + 0,5 = 2,5$ – soma de um número inteiro com decimal positivo.
- $2 + -0,5 = 1,5$ – soma de número inteiro com decimal negativo.
- $-2 + -0,5 = -2,5$ – soma de número negativo com decimal positivo.

Roteiro de Testes

- Objetivo: Descrição detalhada do passo a passo para a execução do sistema, a fim de verificar cada caso de teste identificado na fase anterior.

Para cada caso de teste deve haver um roteiro contendo as seguintes informações:

- Nome do caso de testes;
- Procedimento inicial para determinar onde começa o teste;
- Descrição detalhada contendo:
 - Passos para a execução;
 - Dados de entrada;
 - Resultado esperado;
 - Situação (sucesso ou não);
 - Data da realização;
 - Usuário que realizou o teste;
 - Evidência de realização do teste.

Roteiro de Teste – Exemplo

Caso de teste: consultar cliente com sucesso			
Procedimento inicial: acessar a URL <xxxxx> como usuário administrador, acessar o menu Cadastros, acessar a opção Cliente e clicar em Consultar			
ID	Passo para execução	Dado de entrada	Resultado
1	Sistema exibe tela para pesquisa por nome ou CPF	-	Dados exibidos: campos, nome e CPF
2	Usuário informa CPF válido e clica em Buscar	111.111.111-11	Sistema exibe dados do cliente: nome: xxxx, endereço: xxxxx, cidade: xxxx, estado: xx e país: xxxxx
3	Usuário clica em Nova Busca	-	Tela é limpa, e retorna a tela de pesquisa

Fonte: Livro-texto

Testes Funcionais em Interfaces

- Casos de testes relativos ao comportamento técnico das telas ou interfaces.
- Esses casos de testes são importantes para garantir que a interface faça as verificações necessárias para tornar o *software* mais robusto e confiável com os dados de entrada.

Elemento	Descrição	Tipo/tamanho	Formato	Validação
Campo	Nome	Alfa (40)	Alinhado à esquerda	Pode estar em branco
Campo	CPF	Numérico	111.111.111-1	CPF deve ser válido
Campo	Endereço	Alfa (40)	Alinhado à esquerda	-
Campo	Cidade	Alfa (20)	Alinhado à esquerda	-
Campo	Estado	Alfa (2)	-	UFs válidas do Brasil
Campo	País	Alfa (10)	Fixo	“Brasil”
Botão	Buscar	-	-	Obter dados
Botão	Nova Busca	-	-	Limpa tela
Botão	Voltar	-	-	Retorna ao menu

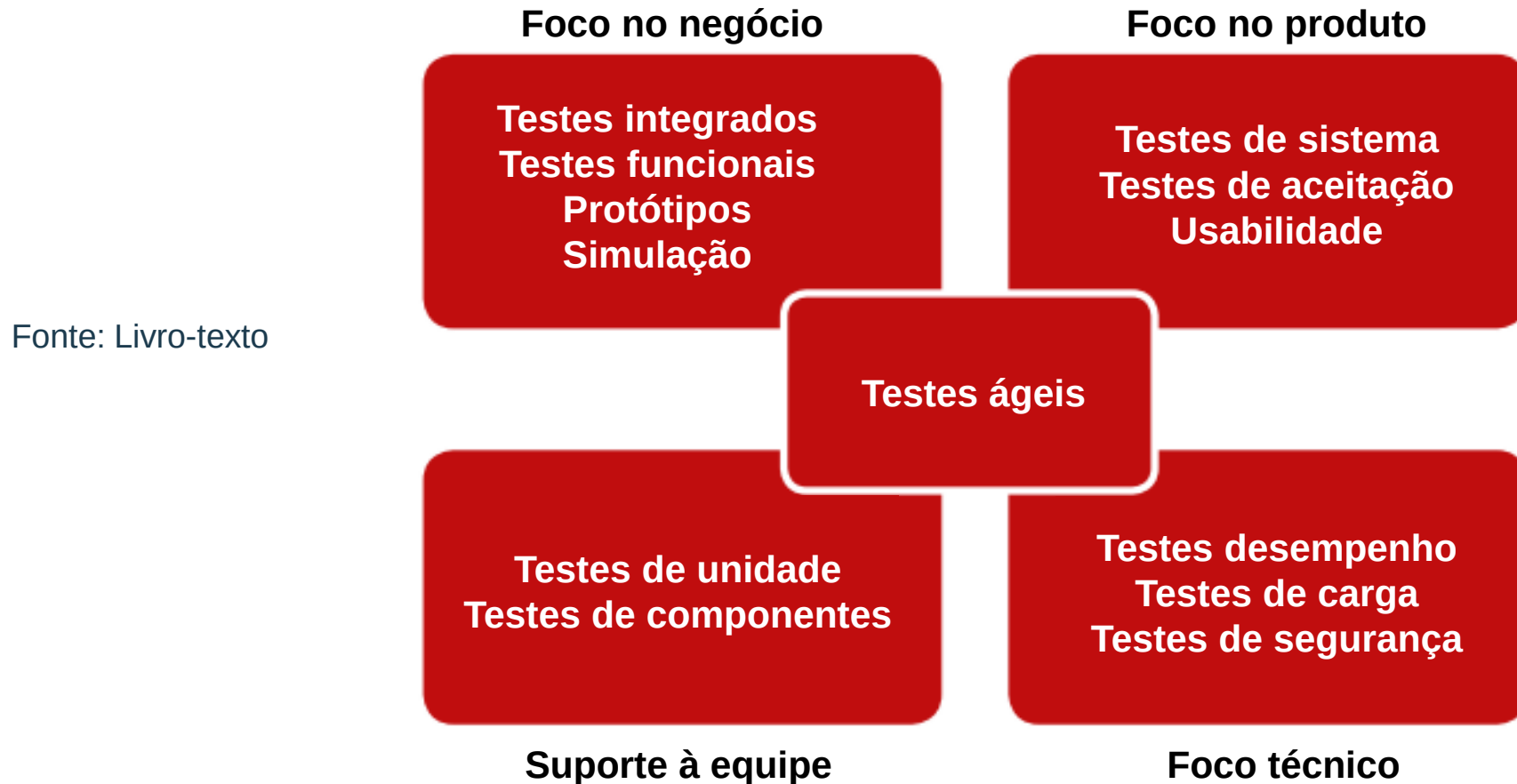
Elemento	Descrição	Situação	Mensagem a ser exibida
Botão	Buscar	Cliente não encontrado	“Cliente não encontrado”

Testes em processos ágeis

- Objetivo: Colaborar com a solução dos defeitos e não apenas apontá-los, por meio da identificação das causas desses defeitos, para que não voltem a acontecer.
- Características:
 - Neste teste, há um compromisso e uma responsabilidade de toda a equipe.
 - Todos devem possuir habilidade para testar e todos trabalham em conjunto, com interação constante durante todo o ciclo de desenvolvimento do *software*.
 - Não há uma fase de testes específica.
 - Os testes são realizados à medida que a codificação termina, e o *feedback* é imediato.

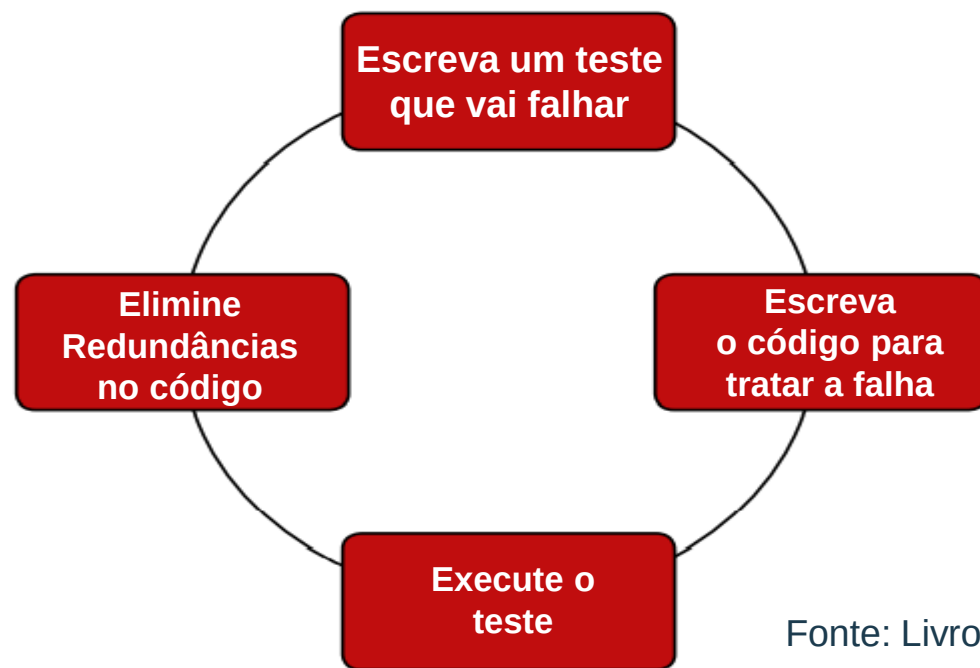
Processos em testes ágeis

- Está baseado nos quadrantes;
- Agrupam os testes em visões;
- Auxiliam na definição do que deve ser feito em cada nível de testes.



Test Driven Development (TDD)

- Técnica utilizada para a realização de testes unitários e de integração.
- Inicia pela identificação dos casos de teste.
- Em seguida, faz-se a escrita do código necessário para atender ao caso de testes.
- É um método para construir *software* direcionado ao programador.
- Objetivo: Melhora a qualidade do código produzido.
- Vem do conceito ágil da Extreme Programming de “testar primeiro que programar”.



Fonte: Livro-texto

Quando concluir os testes?

- Verificar tempo X custo;
- Se o número de defeitos está dentro de uma margem aceitável;
- A frequência em que os defeitos são encontrados é pequena;
- Se houver a garantia de que os testes-caixa foram realizados;
- Todos os casos de testes tiverem sido realizados com sucesso.

Interatividade

Como funciona uma equipe ágil com relação aos testes?

- a) O cliente realiza os testes.
- b) O programador é o tester.
- c) A um *stakeholder* definido pelo time.
- d) Todos devem possuir habilidade para testar e todos trabalham em conjunto, com interação constante durante todo o ciclo de desenvolvimento do *software*.
- e) Não é equipe de teste.

Resposta

Como funciona uma equipe ágil com relação aos testes?

- a) O cliente realiza os testes.
- b) O programador é o tester.
- c) A um *stakeholder* definido pelo time.
- d) Todos devem possuir habilidade para testar e todos trabalham em conjunto, com interação constante durante todo o ciclo de desenvolvimento do *software*.
- e) Não é equipe de teste.

ATÉ A PRÓXIMA!